Universidad Autónoma de Aguascalientes

Ingeniería en Sistemas Computacionales

Programación Web · Grupo 5.º C

Sistema de Contabilidad Web

Primer avance del proyecto

Sprint 1

Integrantes

Landon Enrique Reyes De Lira

Ian Slash Contreras García

Arturo Martínez Diaz De León

Ángel Emmanuel Ortega Sánchez

Marco Antonio Calvillo Durón

Octubre de 2026

Contenido

1. Requisitos y alcance
2. Casos de uso
3. Modelo entidad relación
4. Historias de usuario y criterios de aceptación
5. Arquitectura propuesta
6. Plan de seguridad
7. Plan, responsables y evidencias

Introducción

Este primer avance define los requisitos, el modelo de datos, los casos de uso y la organización necesaria para desarrollar el Sistema de Contabilidad Web. El sistema está dirigido a pequeñas y medianas empresas que necesitan registrar movimientos y consultar estados financieros desde un navegador.

El alcance previsto comprende acceso por empresa y rol, catálogo de cuentas, pólizas balanceadas, consulta de saldos, balance general, estado de resultados, evolución mensual y exportación a PDF. Se utilizará MySQL como base de datos, decisión posterior a la propuesta inicial. Las asignaciones de trabajo son propuestas y deben confirmarse con el equipo; el tablero, las tareas y los pull requests de GitHub siguen pendientes.

1. Requisitos y alcance

Actores

- **Administrador de empresa:** configura la empresa, gestiona integrantes y consulta toda la información contable.
- **Contador:** administra cuentas, registra pólizas y genera reportes de la empresa asignada.
- **Consultor:** consulta saldos y reportes sin modificar datos.

Requisitos funcionales

ID	Requisito	Prioridad	Verificación esperada
RF-01	El sistema debe autenticar a cada usuario antes de permitir acceso a información contable.	Alta	Un visitante no puede consultar recursos protegidos.
RF-02	El sistema debe limitar cada operación a las empresas y permisos asociados con el usuario.	Alta	Un usuario no puede consultar ni modificar datos de otra empresa.
RF-03	El administrador o contador debe poder crear, consultar, actualizar y desactivar cuentas contables.	Alta	El catálogo refleja los cambios válidos sin borrar historial.
RF-04	El contador debe poder registrar una póliza con una o más líneas de cargo y abono.	Alta	La póliza sólo se publica si la suma del debe coincide con la suma del haber.
RF-05	El usuario autorizado debe poder consultar movimientos y saldos por cuenta y periodo.	Alta	La consulta muestra saldo inicial, movimientos y saldo final.

ID	Requisito	Prioridad	Verificación esperada
RF-06	El sistema debe generar el balance general de un periodo con activos, pasivos y capital.	Alta	Se muestran subtotales y la comprobación Activo = Pasivo + Capital.
RF-07	El sistema debe generar el estado de resultados con ingresos, gastos y utilidad o pérdida.	Alta	La utilidad corresponde a ingresos menos gastos del periodo.
RF-08	El sistema debe mostrar la evolución mensual, variación porcentual y promedio histórico de los resultados.	Media	La interfaz presenta valores y gráfica para el intervalo seleccionado.
RF-09	El usuario autorizado debe poder exportar los estados financieros a PDF.	Media	El archivo contiene empresa, periodo, fecha de generación y cifras mostradas.
RF-10	El administrador debe poder invitar, desactivar y asignar roles a integrantes de su empresa.	Media	Los permisos cambian sin afectar a otras empresas.
RF-11	El sistema debe conservar un registro auditable de accesos y acciones contables sensibles.	Alta	Se identifica actor, acción, recurso y fecha sin guardar secretos.

Reglas de negocio

ID	Regla
RN-01	Toda cuenta pertenece a una empresa y tiene código único dentro de ella.
RN-02	Una cuenta se clasifica como activo, pasivo, capital, ingreso o gasto; puede tener un subtipo.
RN-03	Los importes monetarios se almacenan con precisión decimal; no se usan valores de punto flotante binario.
RN-04	Una póliza publicada debe tener al menos dos líneas, importes positivos y total debe igual a total haber.
RN-05	Una póliza publicada no se elimina ni edita directamente; se corrige mediante cancelación o póliza de ajuste.
RN-06	Las cuentas con movimientos se desactivan, no se eliminan.
RN-07	Todos los cálculos de reportes se filtran por empresa y periodo contable.
RN-08	La utilidad o pérdida se calcula como ingresos menos gastos del periodo.
RN-09	El balance general debe indicar si la ecuación contable está cuadrada.
RN-10	Las fechas se almacenan de forma normalizada y se presentan en la zona horaria configurada para la empresa.

Requisitos no funcionales

ID	Requisito	Criterio medible para el MVP
RNF-01	La interfaz debe funcionar en navegadores modernos de escritorio y móvil.	Flujos principales utilizables desde 360 px de ancho y versiones actuales de Chrome, Edge y Firefox.
RNF-02	La interfaz debe ser comprensible y responsiva.	Navegación por teclado, etiquetas de formulario y mensajes de error asociados al campo.
RNF-03	Las respuestas de consulta comunes deben ser oportunas.	El percentil 95 de la API es menor a 1 segundo con el volumen de prueba acordado.

ID	Requisito	Criterio medible para el MVP
RNF-04	Los datos deben persistir en una base relacional.	Reiniciar el servidor no elimina cuentas, pólizas ni integrantes.
RNF-05	El proyecto debe separar rutas, controladores, servicios, acceso a datos, validadores y middlewares.	La revisión de estructura no encuentra lógica de negocio en rutas ni SQL en controladores.
RNF-06	La aplicación debe aplicar defensa en profundidad.	Validación de servidor, autenticación, autorización, aislamiento por empresa, secretos externos y auditoría.
RNF-07	Los errores deben ser seguros y trazables.	El cliente recibe un mensaje controlado y un identificador; el servidor registra el detalle sin datos sensibles.
RNF-08	El desarrollo debe ser reproducible.	Instrucciones y variables de entorno de ejemplo permiten configurar el proyecto sin publicar secretos.

Supuestos y límites

- El producto es una aplicación web; esto está establecido tanto en la rúbrica como en la propuesta.
- La propuesta fija Node.js, Express, HTML, CSS y JavaScript.
- La propuesta original consideraba MySQL o PostgreSQL. El equipo decidió posteriormente utilizar MySQL porque ya lo tiene disponible y posee experiencia previa con este motor.
- La primera versión se orienta a una empresa por membresía, pero un usuario puede pertenecer a más de una empresa.
- El prototipo C++ se usa como referencia conceptual para cuentas, debe, haber, saldos y balance; su archivo de texto no se adopta como persistencia web.

2. Casos de uso

Resumen

ID	Caso de uso	Actor principal	Resultado
CU-01	Iniciar sesión	Cualquier integrante	Sesión autenticada y empresa activa seleccionada.
CU-02	Gestionar integrantes y roles	Administrador	Membresías y permisos actualizados.
CU-03	Administrar catálogo de cuentas	Administrador o contador	Cuenta creada, actualizada o desactivada.
CU-04	Registrar póliza contable	Contador	Póliza balanceada publicada.
CU-05	Consultar movimientos y saldos	Integrante autorizado	Mayor de cuenta filtrado por periodo.
CU-06	Generar balance general	Integrante autorizado	Activos, pasivos, capital y comprobación del balance.
CU-07	Generar estado de resultados	Integrante autorizado	Ingresos, gastos y utilidad o pérdida del periodo.
CU-08	Analizar evolución mensual	Integrante autorizado	Serie mensual, variación y promedio histórico.
CU-09	Exportar estado financiero	Integrante autorizado	PDF con el reporte consultado.

CU-01 Iniciar sesión

Precondición: el usuario tiene una cuenta activa y una membresía vigente.

Flujo principal:

1. El usuario captura correo y contraseña.
2. El servidor valida formato, credenciales, estado del usuario y límite de intentos.
3. El sistema crea una sesión segura.
4. El usuario selecciona una empresa si pertenece a más de una.
5. El sistema muestra el panel correspondiente a su rol.

Alternativas: credenciales inválidas producen un error genérico; una cuenta bloqueada no inicia sesión; una solicitud excesiva recibe limitación temporal.

Postcondición: existe una sesión autenticada y auditable, sin revelar credenciales en registros.

CU-02 Gestionar integrantes y roles

Precondición: sesión activa con rol de administrador en la empresa.

Flujo principal: el administrador consulta integrantes, invita o selecciona uno, asigna un rol permitido y guarda. El servidor verifica que el administrador actúa sobre su propia empresa y registra el cambio.

Alternativas: un contador o consultor recibe 403; un correo inválido recibe 422; no se permite retirar al último administrador activo.

CU-03 Administrar catálogo de cuentas

Precondición: sesión de administrador o contador.

Flujo principal: el usuario captura código, nombre, tipo y subtipo; el servidor valida unicidad y catálogo; la cuenta se crea dentro de la empresa activa.

Alternativas: un código duplicado recibe 409; datos inválidos reciben 422; una cuenta con movimientos se desactiva en lugar de eliminarse.

CU-04 Registrar póliza contable

Precondición: sesión de contador, periodo abierto y cuentas activas de la empresa.

Flujo principal:

1. El contador indica fecha, concepto y líneas de movimiento.
2. Cada línea selecciona una cuenta y captura un cargo o un abono mayor que cero.
3. El servidor valida pertenencia de cuentas, periodo, importes y permisos.
4. El sistema comprueba que total debe es igual a total haber.
5. La póliza se guarda de forma transaccional y se registra en auditoría.

Alternativas: póliza descuadrada, cuenta ajena, periodo cerrado o importe inválido reciben 422 o 403 según corresponda; si falla una línea, no se guarda ninguna.

CU-05 Consultar movimientos y saldos

Precondición: sesión autorizada.

Flujo principal: el usuario elige cuenta y periodo; el sistema valida pertenencia, calcula saldo inicial y muestra movimientos ordenados con saldo acumulado.

Alternativas: una cuenta ajena se trata como recurso no accesible; un intervalo inválido recibe 422.

CU-06 Generar balance general

Precondición: sesión autorizada y fecha de corte válida.

Flujo principal: el sistema agrega cuentas de activo, pasivo y capital, muestra subtotales y compara activo contra pasivo más capital.

Alternativa: si existe descuadre, el reporte se genera con una advertencia visible y queda evidencia para revisión.

CU-07 Generar estado de resultados

Precondición: sesión autorizada y periodo válido.

Flujo principal: el sistema agrega ingresos y gastos, calcula utilidad o pérdida y presenta el detalle por cuenta.

Alternativa: si no existen movimientos, el reporte muestra importes en cero y no inventa datos.

CU-08 Analizar evolución mensual

Precondición: sesión autorizada y al menos un periodo seleccionado.

Flujo principal: el usuario elige un intervalo; el sistema calcula resultado mensual, variación contra el mes anterior y promedio; la interfaz muestra tabla y gráfica.

Alternativa: cuando el mes anterior es cero, el sistema indica que la variación porcentual no aplica.

CU-09 Exportar estado financiero

Precondición: un balance general o estado de resultados ya fue autorizado y generado.

Flujo principal: el usuario solicita PDF; el sistema vuelve a validar permisos, genera el archivo con empresa, periodo, fecha e importes y registra la exportación.

Alternativa: una sesión vencida debe autenticarse otra vez; un reporte ajeno nunca se exporta.

3. Modelo entidad relación

Diagrama lógico

Entidades y atributos

Entidad	Atributos (tipo y clave)
usuario	uuid id_usuario PK; varchar correo UK; varchar clave_hash; varchar nombre; varchar estado; datetime creado; datetime actualizado

Entidad	Atributos (tipo y clave)
empresa	uuid id_empresa PK; varchar razon_social; varchar nombre; varchar zona_horaria; varchar moneda; datetime creado; datetime actualizado
acceso	uuid id_acceso PK; uuid id_usuario FK; uuid id_empresa FK; varchar rol; varchar estado; datetime creado
cuenta	uuid id_cuenta PK; uuid id_empresa FK; varchar codigo; varchar nombre; varchar tipo; varchar subtipo; boolean activa; datetime creado; datetime actualizado
periodo	uuid id_periodo PK; uuid id_empresa FK; date inicio; date fin; varchar estado
poliza	uuid id_poliza PK; uuid id_empresa FK; uuid id_periodo FK; uuid id_usuario FK; date fecha; varchar referencia; varchar concepto; varchar estado; datetime creado
movimiento	uuid id_movimiento PK; uuid id_poliza FK; uuid id_cuenta FK; decimal debe; decimal haber; varchar descripcion
bitacora	uuid id_bitacora PK; uuid id_empresa FK; uuid id_usuario FK; varchar accion; varchar recurso; uuid id_recurso; varchar resultado; datetime fecha

Relaciones

Origen	Destino	Relación
usuario	acceso	tiene
empresa	acceso	incorpora
empresa	cuenta	posee
empresa	periodo	organiza
empresa	poliza	registra
usuario	poliza	crea
periodo	poliza	contiene
poliza	movimiento	detalla
cuenta	movimiento	recibe
usuario	bitacora	ejecuta
empresa	bitacora	conserva

acceso vincula a cada usuario con una empresa y su rol. Cada movimiento es una línea de una poliza y afecta una cuenta. bitacora conserva las acciones relevantes para auditoría. PK identifica la llave primaria, FK una llave foránea y UK un valor único.

Restricciones de integridad

- usuario.correo es único después de normalizar mayúsculas y espacios.
- acceso tiene una restricción única para id_usuario + id_empresa.
- cuenta tiene una restricción única para id_empresa + codigo.
- Todas las relaciones contables deben pertenecer a la misma empresa; no basta con validar que los identificadores existan.

- movimiento.debe y movimiento.haber son decimales no negativos. Exactamente uno debe ser mayor que cero.
- La suma de débitos y créditos se valida en una transacción antes de publicar la póliza.
- Un periodo tiene inicio <= fin; periodos cerrados no admiten nuevas pólizas.
- acceso.rol usa los códigos de la matriz de permisos: ADMIN (administrador), ACCOUNTANT (contador) o VIEWER (consulta).
- cuenta.tipo usa los códigos previstos para activo (ASSET), pasivo (LIABILITY), capital (EQUITY), ingreso (REVENUE) o gasto (EXPENSE).
- Los reportes se calculan a partir de pólizas publicadas; no se persisten cifras derivadas como fuente de verdad.

Correspondencia con el prototipo C++

Prototipo	Modelo web
Cuenta y sus subclases	cuenta con tipo y subtipo validados
Acumulados debe y haber	movimiento, que conserva cada cargo y abono
Contabilidad	Servicios de pólizas y reportes limitados por empresa
ArchivoManager y archivo de texto	Repositorios sobre una base de datos relacional
Resumen financiero	Consultas de balance general y estado de resultados

El diseño web conserva la idea contable del prototipo, pero evita guardar únicamente acumulados: cada movimiento debe quedar trazable.

Implementación prevista en MySQL

- Se utilizará MySQL como motor relacional; la propuesta original que mencionaba MySQL o PostgreSQL queda como antecedente histórico.
- Los importes se implementarán con `DECIMAL`, con precisión y escala definidas en la migración, nunca con `FLOAT` o `DOUBLE`.
- Los identificadores UUID se almacenarán de forma consistente como `CHAR(36)` o `BINARY(16)`. La alternativa se cerrará antes de escribir la primera migración y no se mezclarán ambos formatos.
- Las tablas usarán un motor transaccional que permita claves foráneas y operaciones atómicas para publicar pólizas.
- Los campos de catálogo y estado tendrán restricciones en la aplicación y, cuando resulte apropiado, restricciones compatibles con la versión de MySQL adoptada.
- Los índices compuestos comenzarán por id_empresa en las consultas que aíslan datos por empresa.

La disponibilidad indicada por el equipo no se considera evidencia de configuración: la conexión, versión, usuario de aplicación y migraciones se comprobarán durante el Sprint 2.

4. Historias de usuario y criterios de aceptación

HU-01 Acceso seguro

Como integrante, **quiero** iniciar y cerrar sesión **para** acceder únicamente a la información autorizada.

Criterios de aceptación

- Dado un usuario activo con credenciales válidas, cuando inicia sesión, entonces obtiene una sesión segura y accede a sus empresas.
- Dadas credenciales inválidas, cuando intenta ingresar, entonces recibe un mensaje genérico sin confirmar si el correo existe.
- Dada una ruta protegida sin sesión, cuando se solicita, entonces responde 401.
- Al cerrar sesión, la sesión anterior deja de ser válida.

HU-02 Roles y aislamiento por empresa

Como administrador, **quiero** asignar roles **para** controlar quién consulta o modifica la contabilidad.

Criterios de aceptación

- Un administrador puede invitar, desactivar y cambiar el rol de integrantes de su empresa.
- Un contador y un consultor reciben 403 al intentar administrar integrantes.
- Ningún usuario puede leer o modificar recursos de una empresa sin membresía activa.
- No se puede dejar una empresa sin al menos un administrador activo.

HU-03 Catálogo de cuentas

Como contador, **quiero** administrar cuentas contables **para** clasificar correctamente los movimientos.

Criterios de aceptación

- Se exigen código, nombre y tipo válidos.
- El código no se repite dentro de una empresa, aunque puede existir en otra.
- Una cuenta con movimientos se desactiva en lugar de eliminarse.
- Un consultor puede leer el catálogo, pero no modificarlo.

HU-04 Registro de pólizas

Como contador, **quiero** registrar cargos y abonos **para** conservar movimientos contables consistentes.

Criterios de aceptación

- Una póliza exige fecha, concepto, dos o más líneas y cuentas activas de la empresa.
- Cada línea tiene exclusivamente un cargo o un abono positivo.
- La póliza se publica sólo cuando total debe es igual a total haber.
- Si una validación o escritura falla, no se persiste ninguna parte de la póliza.
- Un periodo cerrado rechaza nuevas pólizas.

HU-05 Movimientos y saldos

Como integrante autorizado, **quiero** consultar el mayor de una cuenta **para** conocer su saldo y movimientos.

Criterios de aceptación

- La consulta acepta una cuenta propia y un periodo válido.
- Se muestran saldo inicial, fecha, referencia, concepto, debe, haber y saldo acumulado.
- El orden es cronológico y estable.
- Una cuenta de otra empresa no revela su existencia ni sus datos.

HU-06 Balance general

Como dueño o contador, **quiero** generar un balance general **para** conocer la posición financiera a una fecha.

Criterios de aceptación

- El reporte agrupa activos, pasivos y capital y muestra subtotales.
- El reporte comprueba Activo = Pasivo + Capital.
- Un descuadre se muestra de forma visible.
- Las cifras provienen sólo de pólizas publicadas de la empresa y fecha seleccionadas.

HU-07 Estado de resultados

Como dueño o contador, **quiero** generar un estado de resultados **para** conocer la utilidad o pérdida de un periodo.

Criterios de aceptación

- El reporte agrupa ingresos y gastos.
- La utilidad o pérdida es ingresos menos gastos.
- Un periodo sin movimientos muestra cero y no un error técnico.
- Las cifras respetan empresa, periodo y permisos.

HU-08 Tendencia mensual

Como dueño, **quiero** comparar resultados mensuales **para** identificar cambios en el desempeño financiero.

Criterios de aceptación

- Se muestran resultado por mes, variación porcentual y promedio del intervalo.
- La gráfica y la tabla representan los mismos valores.
- Si el valor previo es cero, la variación se marca como no aplicable.
- El intervalo inválido se rechaza con 422.

HU-09 Exportación a PDF

Como integrante autorizado, **quiero** exportar un estado financiero **para** compartir o archivar el resultado.

Criterios de aceptación

- El PDF identifica empresa, tipo de reporte, periodo y fecha de generación.
- Los importes coinciden con la vista web.
- El servidor vuelve a verificar autorización al generar el archivo.
- La exportación se registra en auditoría sin incluir información secreta.

Priorización para implementación

Prioridad	Historias	Meta sugerida
Sprint 2	HU-01, HU-03, HU-04 y parte de HU-05	Primer flujo funcional con persistencia y pruebas de integración.
Sprint 3	HU-02, HU-06, HU-07 y completar HU-05	Integración, autorización, despliegue y CI/CD.
Sprint 4	HU-08 y HU-09, correcciones y deuda técnica	Consolidación, E2E y endurecimiento de seguridad.

5. Arquitectura propuesta

Vista general

Flujo de componentes

Componente	Se comunica con
Navegador	Rutas Express
Rutas Express	Middlewares
Middlewares	Controladores
Controladores	Servicios de aplicación
Servicios de aplicación	Repositorios y modelos
Repositorios y modelos	MySQL
Servicios de aplicación	Auditoría

La interfaz web consume una API REST. Las rutas declaran endpoints; los middlewares atienden preocupaciones transversales; los controladores traducen HTTP; los servicios contienen reglas de negocio y transacciones; los repositorios concentran el acceso a datos.

Organización prevista

```
src/
  app.js
  config/
  routes/
    auth.routes.js
    memberships.routes.js
    accounts.routes.js
    journal-entries.routes.js
    reports.routes.js
  controllers/
  services/
  repositories/
  models/
  validators/
  middlewares/
```

```
    authenticate.js
    authorize.js
    company-context.js
    validate.js
    rate-limit.js
    error-handler.js
  utils/
tests/
  integration/
  e2e/
```

Responsabilidades

Capa	Sí debe hacer	No debe hacer
Rutas	Declarar método, URL, middlewares y controlador.	Contener reglas contables o consultas SQL.
Middlewares	Autenticar, autorizar, establecer empresa, validar solicitud y manejar errores.	Calcular reportes o decidir reglas de pólizas.
Controladores	Leer solicitud, invocar un servicio y construir la respuesta HTTP.	Acceder directamente a la base o duplicar validaciones de negocio.
Servicios	Aplicar reglas, coordinar repositorios y controlar transacciones.	Depender de objetos específicos de Express.
Repositorios y modelos	Persistir y consultar datos con filtros por empresa.	Decidir permisos del usuario.
Validadores	Comprobar forma, tipos, límites y campos requeridos.	Sustituir autorización o integridad transaccional.

Endpoints iniciales

Método y ruta	Propósito	Roles
`POST /api/v1/auth/sessions`	Iniciar sesión.	Público
`DELETE /api/v1/auth/sessions/current`	Cerrar sesión.	Autenticado
`GET /api/v1/companies/:companyId/memberships`	Consultar integrantes.	ADMIN
`PATCH /api/v1/companies/:companyId/memberships/:id`	Cambiar rol o estado.	ADMIN
`GET /api/v1/companies/:companyId/accounts`	Consultar catálogo.	ADMIN, ACCOUNTANT, VIEWER
`POST /api/v1/companies/:companyId/accounts`	Crear cuenta.	ADMIN, ACCOUNTANT
`PATCH /api/v1/companies/:companyId/accounts/:id`	Actualizar o desactivar cuenta.	ADMIN, ACCOUNTANT
`POST /api/v1/companies/:companyId/journal-entries`	Crear póliza.	ADMIN, ACCOUNTANT
`GET /api/v1/companies/:companyId/accounts/:id/ledger`	Consultar mayor.	ADMIN, ACCOUNTANT, VIEWER

Método y ruta	Propósito	Roles
`GET /api/v1/companies/:companyId/reports/balance-sheet`	Generar balance general.	ADMIN, ACCOUNTANT, VIEWER
`GET /api/v1/companies/:companyId/reports/income-statement`	Generar estado de resultados.	ADMIN, ACCOUNTANT, VIEWER
`GET /api/v1/companies/:companyId/reports/monthly-trend`	Consultar tendencia.	ADMIN, ACCOUNTANT, VIEWER

Convenciones HTTP

- `200` para consultas o cambios exitosos, `201` para creación y `204` para cierre de sesión sin contenido.
- `400` para solicitud mal formada, `401` sin autenticación, `403` sin permiso, `404` cuando el recurso no es accesible, `409` para conflicto de unicidad y `422` para reglas de validación.
- Las respuestas de error tienen `code`, `message` y `requestId`; no incluyen trazas ni SQL.
- Todas las rutas se versionan bajo `/api/v1`.

Decisiones registradas

Decisión	Estado	Motivo
Aplicación web con API REST	Aceptada	Coincide con la propuesta y la materia.
Node.js y Express.js	Aceptada	Está especificado en la propuesta.
MySQL como base de datos relacional	Aceptada	La propuesta admitía MySQL o PostgreSQL; el equipo eligió MySQL porque ya lo tiene disponible y posee experiencia previa en varios proyectos.
Pólizas con líneas en vez de acumulados editables	Aceptada	Mejora trazabilidad, integridad y auditoría respecto del prototipo.
Aislamiento por id_empresa	Aceptada	Previene acceso a datos ajenos y habilita múltiples empresas; coincide con la llave del modelo ER.

La disponibilidad de MySQL proviene de la confirmación del equipo. Su instalación, versión y configuración todavía no se han verificado como parte de este avance documental.

6. Plan de seguridad

Objetivos

- Proteger credenciales, sesiones e información financiera.
- Impedir acceso horizontal entre empresas y elevación de privilegios.
- Validar toda entrada en el servidor.
- Mantener secretos fuera del código y del repositorio.
- Producir evidencia reproducible de controles y pruebas.

Roles y permisos

Operación	ADMIN	ACCOUNTANT	VIEWER
Administrar integrantes y roles	Sí	No	No
Crear y modificar catálogo	Sí	Sí	No
Registrar pólizas	Sí	Sí	No
Consultar movimientos y reportes	Sí	Sí	Sí
Exportar reportes	Sí	Sí	Sí

La autorización verifica simultáneamente identidad, membresía activa, empresa del recurso y rol. Recibir un identificador válido nunca es suficiente para autorizar.

Validaciones previstas

Entrada	Validación de servidor
Correo	Normalización, longitud máxima y formato válido.
Contraseña	Longitud mínima, máximo razonable y comparación mediante hash resistente; nunca se registra.
Identificadores	UUID válido y pertenencia a la empresa activa.
Código de cuenta	Caracteres permitidos, longitud, unicidad por empresa y tipo permitido.
Importes	Decimal finito, positivo, escala monetaria y límite máximo documentado.
Póliza	Dos o más líneas, una columna por línea, cuentas activas y suma debe igual a suma haber.
Fechas y periodos	Formato estricto, rango válido y periodo abierto.
Texto libre	Longitud máxima; salida codificada para evitar inyección de HTML.
Paginación	Enteros con límites máximos para evitar consultas abusivas.

Las consultas se parametrizan mediante el mecanismo del ORM o controlador SQL elegido. No se concatena entrada del usuario en SQL.

Autenticación y sesión

- Hash de contraseñas con un algoritmo diseñado para contraseñas y parámetros configurables.
- Cookies de sesión `HttpOnly`, `Secure` en producción y `SameSite` apropiado; si se usan tokens, los de acceso serán breves y los de renovación serán revocables.
- Protección CSRF cuando la autenticación dependa de cookies.
- Limitación de intentos de inicio de sesión y de endpoints costosos.
- Invalidación de sesión al cerrar sesión, desactivar al usuario o cambiar credenciales.
- Política CORS restringida a orígenes configurados; nunca reflejar cualquier origen con credenciales.

Secretos y configuración

- Los secretos se entregan mediante variables de entorno o un almacén del proveedor de despliegue.
- Se versiona sólo un `.env.example` sin valores reales.

- `.env`, llaves, volcados, respaldos y archivos de credenciales se excluyen con `.gitignore`.
- Los registros y mensajes de error no incluyen contraseñas, cookies, tokens, cadenas de conexión ni datos financieros completos.
- Si un secreto se expone, se revoca y rota; borrar el archivo del último commit no es suficiente.

Registro y auditoría

Se auditan inicio y cierre de sesión, fallos relevantes, cambios de rol, altas o bajas de cuentas, publicación o cancelación de pólizas y exportaciones. Cada evento registra fecha, usuario, empresa, acción, recurso, resultado y `requestId`, sin secretos.

Casos de prueba de seguridad planeados

ID	Caso	Resultado esperado
SEG-01	Solicitar una ruta protegida sin sesión.	401 sin datos.
SEG-02	Un VIEWER intenta crear una cuenta.	403 y sin cambio persistido.
SEG-03	Usuario de empresa A solicita una cuenta de empresa B.	404 o 403 consistente, sin filtrar datos.
SEG-04	Se envía un `companyId` ajeno en URL o cuerpo.	Rechazo y evento auditable.
SEG-05	Código de cuenta duplicado en la misma empresa.	409; en otra empresa puede ser válido.
SEG-06	Póliza con debe distinto de haber.	422 y ninguna línea persistida.
SEG-07	Póliza con valor negativo, texto en importe o valor fuera de rango.	422 y mensaje de campo controlado.
SEG-08	Entrada con payload de SQL o HTML.	Se trata como dato; no ejecuta SQL ni script.
SEG-09	Repetir intentos de inicio de sesión.	Limitación temporal sin revelar cuentas válidas.
SEG-10	Token o sesión vencida.	401; no se renueva de forma indefinida.
SEG-11	Error interno forzado.	500 con `requestId`, sin traza ni secreto en la respuesta.
SEG-12	Buscar patrones de secretos en repositorio e historial.	Ningún secreto real encontrado.

Criterio de salida por sprint

- **Sprint 1:** este plan, roles, validaciones, manejo de secretos y casos de prueba están documentados.
- **Sprint 2:** los controles asociados al flujo implementado funcionan y tienen pruebas de integración.
- **Sprint 3:** autenticación, autorización, aislamiento, despliegue y pipeline se verifican en un entorno accesible.
- **Sprint 4:** se corrigen hallazgos, se ejecutan pruebas E2E y se reduce deuda de seguridad.
- **Final:** no existen fallas críticas conocidas y la suite completa alcanza al menos 90% de cobertura.

7. Plan, responsables y evidencias

Meta del Sprint 1

Presentar una planeación coherente y trazable que permita iniciar el desarrollo del flujo de autenticación, catálogo y pólizas en el Sprint 2, con responsabilidades, criterios de aceptación y controles de seguridad definidos.

Responsabilidades propuestas

La rúbrica exige tareas con responsables y evidencia de la contribución de cada integrante, pero no prescribe cargos ni esta distribución nominal. Landon eligió cuatro tareas técnicas; las ocho tareas restantes se repartieron al azar entre los otros cuatro integrantes, dos para cada uno. El equipo debe confirmar la propuesta.

Integrante	Responsabilidad principal en Sprint 1	Evidencia
Landon Enrique Reyes De Lira	Modelo ER, decisión de MySQL, arquitectura y endpoints	Modelo de datos, decisión registrada, diseño de capas y contratos preliminares de API.
Ian Slash Contreras García	Requisitos e historias de usuario	Requisitos revisados e historias con criterios de aceptación.
Arturo Martínez Diaz De León	Casos de uso y coordinación de pull requests	Casos de uso revisados y pull requests de documentación.
Ángel Emmanuel Ortega Sánchez	Alcance del MVP y tablero de GitHub	Alcance acordado e issues con responsables.
Marco Antonio Calvillo Durón	Seguridad y coordinación de la revisión	Matriz de permisos, casos de prueba y recorrido de la evidencia.

Backlog listo para crear como issues

ID	Tarea	Responsable propuesto	Criterio de terminado	Estado local
S1-01	Confirmar alcance del MVP y exclusiones	Ángel	El equipo aprueba alcance y pendientes en una minuta o issue.	Documentado
S1-02	Revisar requisitos funcionales y no funcionales	Ian	Cada requisito tiene ID, prioridad y verificación.	Documentado
S1-03	Validar casos de uso contra la propuesta	Arturo	Actores, precondiciones, flujos y alternativas son coherentes.	Documentado
S1-04	Revisar modelo ER y restricciones	Landon	Entidades, llaves, relaciones y reglas de pólizas están definidas.	Documentado
S1-05	Registrar la decisión de utilizar MySQL	Landon	La arquitectura, modelo y plan identifican MySQL y justifican la elección.	Documentado
S1-06	Revisar arquitectura por capas	Landon	Rutas, controladores, servicios, repositorios y middlewares tienen límites claros.	Documentado

ID	Tarea	Responsable propuesto	Criterio de terminado	Estado local
S1-07	Revisar endpoints y códigos HTTP	Landon	Cada flujo del MVP tiene contrato preliminar y roles.	Documentado
S1-08	Revisar historias y aceptación	Ian	Las historias prioritarias son comprobables y trazables.	Documentado
S1-09	Revisar amenazas y pruebas de seguridad	Marco	Validaciones, roles, secretos y casos negativos están definidos.	Documentado
S1-10	Crear tablero e issues en GitHub	Ángel	Todos los items existen en GitHub con responsable, criterio y fecha.	Pendiente externo
S1-11	Coordinar ramas y pull requests de documentación	Arturo	Todos contribuyen; cambios revisados e integrados a develop mediante PR.	Pendiente externo
S1-12	Coordinar demostración del Sprint 1	Marco	Todos presentan; se puede recorrer evidencia y explicar decisiones en la revisión.	Pendiente

Definición de terminado del Sprint 1

- Los requisitos, casos, modelo ER e historias son coherentes entre sí.
- Cada historia priorizada tiene criterios de aceptación observables.
- El diseño separa rutas, controladores, servicios, modelos/repositorios y middlewares.
- El plan de seguridad cubre validación, roles, secretos y pruebas negativas.
- Cada tarea tiene una persona responsable y un resultado esperado.
- El tablero de GitHub refleja el estado real y conserva evidencia de contribuciones.
- Las decisiones pendientes están visibles y no se presentan como resueltas.

Flujo de Git

- `main`: versiones estables y entregas.
- `develop`: integración del sprint.
- `feature/<id>-<descripcion>`: una tarea o funcionalidad concreta.
- `release/`: preparación de una versión evaluable cuando corresponda.
- `hotfix/`: correcciones urgentes sobre una versión estable.

Cada cambio entra mediante pull request hacia `develop`, con al menos una revisión del equipo. Los commits deben ser pequeños, oportunos y describir el cambio, por ejemplo `docs: define modelo ER del sprint 1`.

Evidencias para la revisión

Evidencia	Cómo mostrarla
Planeación	Recorrer el índice, trazabilidad, backlog y definición de terminado.
Coherencia	Seleccionar una historia y seguirla hacia requisito, caso, entidad, endpoint y prueba.
Arquitectura	Explicar el diagrama y por qué la lógica no vive en rutas ni controladores.
Seguridad	Demostrar matriz de roles, aislamiento por empresa, validaciones y casos SEG-01 a SEG-12.
GitHub	Mostrar tablero, issues asignados, ramas, commits y pull requests reales.
Aportación individual	Filtrar issues, commits y revisiones por integrante.

Riesgos y mitigaciones

Riesgo	Impacto	Mitigación
Usar convenciones inconsistentes para UUID, decimales o transacciones en MySQL	Provoca migraciones incompatibles o errores contables.	Definir precisión monetaria, representación UUID y versión objetivo antes de la primera migración.
Mantener la URL de repositorio de ejemplo	Falta evidencia verificable.	Registrar la URL real en README, propuesta y tablero.
Asignaciones no confirmadas	Trabajo duplicado o sin dueño.	Confirmar tabla de responsables y actualizar issues.
Copiar el modelo acumulado del prototipo C++	Se pierde trazabilidad de movimientos.	Persistir pólizas y líneas; calcular saldos.
Autorizar sólo por identificador	Riesgo crítico de acceso a datos ajenos.	Filtrar por empresa y verificar membresía en cada servicio y repositorio.
Commits hechos al final del sprint	No demuestran avance ni contribución.	Integrar cambios pequeños durante el sprint mediante PR.

Pendientes externos

- URL real del repositorio de GitHub.
- Tablero, issues, ramas, commits y pull requests; no pueden demostrarse sólo con archivos locales.
- Confirmación de responsables y fechas por el equipo.
- Verificación de la versión y configuración local de MySQL al iniciar el Sprint 2.